CONTENTS

Advanced | Agentic software engineering | IBM Technology

AI Code Refactoring: Letting a Probabilistic System Edit Production Code Safely

Understand the loop, the verification steps and the deterministic tooling that make autonomous refactoring trustworthy.

For engineers evaluating or building autonomous refactoring into their workflow · 26 July 2026

Watch the source video | Read the condensed version | Read the highlights

BEFORE YOU START

- Working experience with a real codebase and a test suite
- Familiarity with version control and CI
- Comfort reading Python

Overview

A large language model is a probabilistic guessing machine, and these machines are now being pointed at production code. The obvious question is whether that is safe.

The answer turns out not to be about the model at all. Refactoring means changing a program's internal structure while leaving its external behaviour identical, and 'identical' is a checkable property — you have tests, a build, a type checker and a diff. Agentic refactoring is safe to the extent that a guess has to survive those checks before it reaches the codebase, and unsafe to the extent that it does not.

So the interesting content is the loop: plan, read, search, report, approve, snapshot, patch, verify, roll back on failure and go round again. Two details sharpen it further. The patch step can skip probabilistic generation entirely by operating on a syntax tree, making common transformations deterministic. And every accept, reject, pass and fail is a training signal, so the suggestions improve over time.

KEY TAKEAWAYS

- ✓ Refactoring changes internal structure while leaving external behaviour identical — the second half is what makes it verifiable.
- ✓ The three canonical motives are readability, removing duplication and lowering complexity; skipping them compounds into technical debt.
- ✓ Inline refactoring works alongside a developer on small local edits; agentic refactoring is handed a goal and works the whole codebase autonomously.
- ✓ The danger is a probabilistic system deleting things that look unused but exist for a reason nobody wrote down.
- ✓ The guard rail is a loop: plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure.
- ✓ Human approval sits between the report and any edit, so guesses never reach the codebase unreviewed.
- ✓ Because verification is just tests and builds, the whole loop slots into CI/CD like another developer.
- ✓ Patching via a syntax tree makes transformations like rename deterministic rather than probabilistic.

01 Refactoring: change the inside, preserve the outside

0:00

> Take a program's internal structure and change it, while to the outside world the thing still looks exactly the same.

Plain code refactoring means changing a program's internal structure without changing its external behaviour. You rearrange the inside; to the outside world it still looks the same.

That second clause is the entire discipline, and it is what separates refactoring from rewriting. Rewriting is also allowed to change what the code does. Refactoring is not, and because it is not, correctness becomes a checkable property rather than a matter of judgement. You are not asking 'is this new code good?' — you are asking 'does this behave identically to what was there before?', and that is a question tests, types and builds can answer.

It is worth being precise about what 'external behaviour' covers, because it is broader than the return value. The public API and its types. Side effects: what gets written, sent, logged, cached. Errors and exceptions raised. Performance characteristics, at least to the extent anything depends on them. Concurrency behaviour. A refactoring that preserves return values while changing what a function writes to the database has changed external behaviour, whatever it looks like locally.

The practical consequence sets up everything after it. Verifiability depends on having a way to detect a behaviour change, which for most codebases means the test suite. Refactoring a well-tested module is a safe, mechanical activity. Refactoring an untested one is a rewrite you are performing with your eyes

closed — and that is true whether a human or an agent is doing it.

KEY POINTS

- → Refactoring changes internal structure and preserves external behaviour.
- → Preservation is what makes correctness checkable rather than a judgement call.
- → External behaviour includes side effects, errors, types and timing — not just return values.
- → Verifiability depends on being able to detect a behaviour change, which usually means tests.
- → Refactoring untested code is rewriting, whoever performs it.

The invariant, stated as a test

This is the shape of a characterisation test: it does not assert that the behaviour is correct, only that it has not changed. For code you are about to restructure, that is exactly the right assertion, and it can be generated from recorded production inputs.

```
@pytest.mark.parametrize("case", RECORDED_INPUTS)
def test_behaviour_unchanged(case, snapshot):
    result = process(case.input)

    assert result == snapshot(case.id)          # same return value
    assert case.side_effects == captured_writes()   # same writes
    assert case.raised == captured_exceptions()     # same errors

# it does not claim the behaviour is right. it claims it is the same,
# which is the only property a refactoring is allowed to preserve.
```

What counts as external

The left column is free to change. Anything in the right column changing means it was not a refactoring, whatever it was called in the pull request.

```
internal — free to change        external — must not change
─────────────────────────────────────────────────────────────
variable and local names         public function signatures
function decomposition           return values and their types
file and module layout           exceptions raised
control flow shape               database writes, network calls
comments                         emitted logs others depend on
private helpers                  observable ordering and timing
```

02 Why bother, and what happens if you do not

0:43

> Readability, removing duplication, lowering complexity. Skip them long enough and it compounds into technical debt.

If the external view does not change, why do it at all? Three motives cover most cases.

Readability. A variable called `temp2` tells anyone reading the code essentially nothing. Renaming it to something a human can understand — `customer_state` — is refactoring for readability. The program is identical; the cost of understanding it has dropped.

Removing duplication. A block of logic gets copied and pasted several times over the years, and then it turns out to contain a bug. Now the bug has to be fixed in several places, and whoever fixes it has to find all of them. Pulling the logic into one place means fixing it once. Duplication converts one bug into several, and the multiplication is silent.

Lowering complexity. A very long function doing half a dozen different jobs is hard to reason about, hard to test and hard to change safely. Breaking it into pieces that each do one thing is refactoring for complexity.

Skip this cleanup for long enough and it compounds into technical debt: changes take longer than they should, and even simple fixes turn out not to be simple. The debt metaphor is apt in one specific way — the cost is not the mess itself, it is the interest, paid by every future change that has to route around it. Refactoring is repayment, and the reason it keeps getting deferred is that the interest is invisible in any single sprint.

KEY POINTS

- → Readability: names that carry meaning reduce the cost of every future read.
- → Duplication: one copied bug becomes several, and nobody knows how many.
- → Complexity: a function doing six jobs is hard to test and unsafe to change.
- → Technical debt is the compounding cost of skipping this work.
- → The cost is the interest — paid by every future change, invisible in any single sprint.

The three canonical moves

Each preserves behaviour exactly and each is mechanical enough to automate reliably — which is precisely why refactoring was the first place agentic coding tools became genuinely useful.

```
# 1. rename for readability
-  temp2 = fetch(o.cid).st
+  customer_state = fetch(order.customer_id).state

# 2. remove duplication — the same 20 lines existed in 4 files
+  def normalise_address(raw): ...
-  <the same block, inlined, four times, one of them subtly different>

# 3. lower complexity — one 300-line function doing six jobs
+  def validate(order): ...
+  def price(order): ...
+  def apply_discounts(order): ...
+  def charge(order): ...
-  def process_order(order):   # 300 lines, six responsibilities
```

How duplication multiplies a bug

The fourth copy is the interesting one. It drifted at some point, so a blind find-and-replace fixes three sites and quietly changes the behaviour of the fourth — which is why deduplication needs the search step described later, not a text match.

```
billing/invoice.py:112     if days > 28: ...      ← the bug
billing/refund.py:87       if days > 28: ...      ← same bug, copy 2
reports/monthly.py:203     if days > 28: ...      ← same bug, copy 3
export/legacy.py:41        if days >= 28: ...     ← drifted. is this a
                                                    fourth bug, or is it
                                                    deliberate?

# nobody knows without reading the history. this is the interest payment.
```

03 Inline versus agentic refactoring

2:50

> One works next to you on small local edits. The other is handed a goal and works the whole codebase on its own.

Much of refactoring comes down to pattern recognition — the same tired problems appearing over and over in a codebase — and pattern recognition is what these models are good at. That gives two shapes of tool.

Inline refactoring happens inside an IDE or code editor, working alongside the developer as they type. It might spot a clumsy variable name like `temp2` and suggest a better one, or the developer highlights a block and the tool offers to pull it out into its own function. Small, local fixes, made in the editor, with the developer's attention already on that code.

Agentic refactoring is different in kind. Instead of suggesting one edit at a time, the tool is handed a goal — something concrete like 'upgrade this library to the latest version', or something considerably vaguer like 'clean up this whole module'. With the goal set, it works the problem across the entire codebase on its own. That is the distinguishing property: it is autonomous. It takes advantage of an agent's ability to pattern match and to hold a large amount of code in context while it makes its changes.

The difference that matters operationally is the review surface. Inline refactoring is reviewed continuously and implicitly — the developer is looking at the code as the suggestion appears, and rejecting one costs a keystroke. Agentic refactoring produces a large change set that nobody watched being made, so review is a separate, deliberate act performed after the fact. That shift is where the risk enters, and it is why everything that follows is about structuring that review.

KEY POINTS

- → Refactoring is largely pattern recognition, which is what these models do well.
- → Inline refactoring: small local edits in the editor, alongside the developer.
- → Agentic refactoring: given a goal, works the whole codebase autonomously.
- → Goals range from precise ('upgrade this library') to vague ('clean up this module').
- → The real difference is the review surface: continuous and implicit, versus deferred and deliberate.

Two tools, two working modes

The last row is the one that determines how much machinery you need around the tool. A rejected inline suggestion costs a keystroke; a bad agentic change set costs a review of hundreds of files.

```
                   inline                   agentic
──────────────────────────────────────────────────────────────────
input              cursor position          a goal, in English
scope              one expression, one file the whole repository
autonomy           suggests, you accept     plans and executes
duration           instant                  minutes to hours
context held       the current file         project and dependencies
review             continuous, implicit     deferred, deliberate
cost of a bad edit one keystroke            a change set to audit
```

The goals people actually give it

Precision of the goal maps directly onto how much the report stage matters. A precise goal has a checkable definition of done; a vague one is really a request for a proposal, and should be treated as one.

```
precise, checkable
  "upgrade requests from 2.28 to 2.32 and fix any breaking changes"
  "replace every use of the deprecated get_user_v1 with get_user"
  "add type annotations to everything in billing/"

vague, needs a report first
  "clean up this whole module"
  "reduce complexity in the checkout flow"
  "make this more testable"

# vague goals are proposals to review, not instructions to execute
```

04 The risk: a probabilistic system editing hundreds of files

4:55

> It predicts likely-looking code, so it may tidy away something that looked unused but was there for a reason.

This is where the concern from the start of the story returns. A guessing machine sweeping through hundreds of files, largely unsupervised, predicting likely-looking code.

The failure mode is specific. It might tidy up something that looked unused but was actually there for a reason — an obscure function that did something with leap years, say. You find out you needed it the next time the calendar ticks over to 29 February, and by then it is far too late.

What makes this example so good is that the reasoning was locally correct. The function had no visible callers. Nothing referenced it in the obvious places. Removing dead code is a legitimate refactoring. Every step was sound, and the conclusion was still wrong, because the justification for keeping it existed outside the code — in a calendar, in an incident three years ago, in someone's memory.

That generalises into the category worth watching for: things that look unreferenced to static analysis but are reached at runtime, or matter only rarely. Dynamic dispatch and reflection. Entry points invoked by a scheduler, a queue consumer, a CLI or a webhook. Code behind a feature flag that is currently off. Compatibility shims for old clients. Error paths that have not fired yet. Anything seasonal.

And this risk is not unique to AI — a human doing a large cleanup makes the same mistake. What is different is the volume and the speed. A person removes three functions and thinks about each; an agent removes forty across a hundred files in a few minutes, and the review is the only thing standing between that and production. So the mitigation is not to make the model more careful. It is to make sure a guess cannot become a commit without passing something that checks.

KEY POINTS

- → The model predicts likely-looking code; it does not know why something exists.
- → The leap-year example is locally correct reasoning reaching a wrong conclusion.
- → Watch for code reached by reflection, schedulers, queues, flags, shims and rare error paths.
- → The risk is not new — the volume and speed are.
- → The mitigation is verification before commit, not a more careful model.

Correct reasoning, wrong answer

Reconstruct the agent's chain and every step is defensible. The missing information was never in the repository, which is why no amount of reading the code would have prevented it — and why the loop needs steps that are not reading the code.

```
def adjust_for_leap_year(date, term_days):
    """Contract terms crossing 29 Feb need a day added."""
    ...

# the agent's reasoning:
#   1. grep for adjust_for_leap_year → one definition, no calls
#   2. no tests reference it
#   3. no imports elsewhere in the repo
#   4. removing unused code is a valid refactoring
#   → remove it

# what it could not see:
#   called via getattr(rules, f"adjust_for_{rule_name}") in a config-driven
#   dispatcher, where rule_name comes from a YAML file
```

The dead-code checklist

Run these before accepting any deletion, from an agent or a person. Static analysis answers the first line and nothing below it, which is exactly where the leap-year function lived.

```
☐ referenced by getattr / reflection / dynamic import?
☐ an entry point for cron, a queue consumer, a CLI, a webhook?
☐ behind a feature flag that is currently off?
☐ a compatibility shim for clients you do not control?
☐ registered by a decorator or a plugin system?
☐ referenced only from config, YAML, or the database?
☐ an error path that has not fired yet?
☐ seasonal — quarter end, year end, leap day?

# grep answers line 1. everything below it needs the search stage.
```

05 The loop, part one: plan, read, search, report

5:37

Agentic refactoring manages risk by working as a cycle. The first four stages build

> context and produce findings rather than edits.

Agentic refactoring manages the risk by working as a loop. The agent moves through the same steps in order, round and round, until the job is done. Different tools vary in the details, but the shape holds. At the centre sits the goal the human provided.

It starts with plan. The agent works out an ordered list of what it needs to change and in what order, and keeps updating that plan as it learns more.

To make the plan it has to read: open the files involved and map what is in them — the functions, the classes, how they fit together. This is structural comprehension rather than reading for its own sake.

Then search. It searches the rest of the project, and the libraries the project depends on, for related code. If a function is called from a number of places, the search stage is what finds all of those call sites. This is the stage that would have saved the leap-year function, and it is the reason searching dependencies matters: a symbol referenced only from a framework's registration decorator looks unused from inside your own source.

With context established, the agent moves to report. It comes back with findings ranked from high to low priority, each with its own reasoning. A useful finding reads like: this function is ignoring exceptions, that will hide failures in production, and here is the suggested fix.

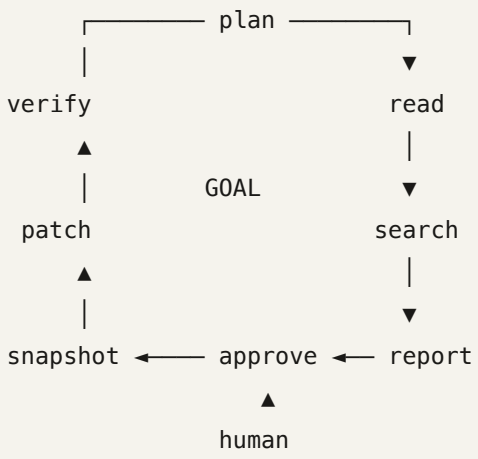
The critical property of these four stages is that none of them modify anything. The agent has read the codebase and produced a document. Everything so far is reversible by closing a tab, and the human has not spent any attention yet.

KEY POINTS

- → The loop runs plan → read → search → report, with the goal at the centre.
- → Plan produces an ordered list of changes and is revised as the agent learns.
- → Read maps structure: functions, classes and their relationships.
- → Search covers the rest of the project and its dependencies to find all references.
- → Report ranks findings by priority, each with explicit reasoning.
- → None of these four stages modifies anything — the output is a document.

The loop

Read it as a cycle rather than a pipeline. A failure at verify does not abort the job — it returns to plan with new information, which is what makes the agent able to recover rather than merely stop.




```
# read / search / report change nothing.
# nothing is written until after approve.
```

What a good finding looks like

Three parts: what, why it matters in production, and the concrete fix. A finding without the middle part cannot be prioritised, and a report of unprioritised findings is what makes humans approve in bulk without reading.

```
[HIGH]  billing/charge.py:88
  Bare `except: pass` around the payment capture call.
  A gateway timeout is swallowed, the order is marked paid, and no
  alert fires. This silently loses revenue and is invisible in logs.
  Fix: catch GatewayTimeout specifically, log with the order id,
       and re-raise so the retry queue picks it up.

[LOW]   reports/monthly.py:12
  Unused import `datetime`.
  No runtime effect. Cosmetic.
```

06 The loop, part two: approve, snapshot, patch

7:44

> The human re-enters here, choosing which findings to fix. Then a snapshot is taken so

> everything can be undone, and only then does code change.

Up to this point the human has done almost nothing beyond setting the goal — the process is autonomous. This is where they re-enter, by picking which findings the agent should actually fix. Some level of human approval sits here.

The placement is the whole design. Approval comes after the analysis and before any edit, which means the expensive autonomous work happens without supervision while the irreversible part does not. The human is reviewing a ranked list with reasoning, not a diff across two hundred files — a far smaller and better-shaped review task.

Once the selection is approved, the agent creates a snapshot of the codebase as it stands. That is the starting state, and it exists so the changes can be undone. In practice this is a branch or a commit, and it matters that it is taken automatically rather than relying on the developer having a clean working tree.

Then it patches: applying changes to the specific lines that need changing, carrying all the context gathered earlier in the loop. That context is what distinguishes this from a find-and-replace — the agent knows which call sites it found in the search stage and which of them are genuinely the same pattern.

One practical note on scope. The value of approval collapses if the change set is too large to review, and a fifty-file diff gets approved in bulk by a tired human at five o'clock. Better to run the loop repeatedly with narrow goals and small, reviewable change sets than once with a broad one. The loop is cheap to run again; a rubber-stamped review is not.

KEY POINTS

- → Human approval sits after analysis and before any modification.
- → The human reviews a ranked list with reasoning, not a large diff.
- → A snapshot is taken automatically so every change can be undone.
- → Patching carries the context gathered during read and search, unlike find-and-replace.
- → Keep change sets small — approval is worthless if the diff is too large to read.

Approval as a selection, not a yes/no

Selecting a subset is what makes the review real. A single accept-everything button trains the human to click it, which removes the guard rail while leaving it visible in the diagram.

```
Findings for goal: "clean up billing/"

  [x] HIGH  charge.py:88     bare except swallows gateway timeouts
  [x] HIGH  invoice.py:112   duplicated day-count logic (4 sites)
  [ ] MED   refund.py:203    extract 180-line function      ← not now
  [x] LOW   monthly.py:12    unused import
  [ ] LOW   legacy.py:41     remove apparently unused helper ← verify first

  → 3 of 5 approved. the agent patches only those.
```

The snapshot has to be automatic

Taken by the tool, not left to the developer's habits. The value of the snapshot is that it exists on the run where something goes wrong, which is exactly the run where nobody remembered to make one.

```
def begin_patching(goal):
    assert_clean_worktree()                       # refuse to start otherwise
    base = git.rev_parse("HEAD")
    branch = git.create_branch(f"refactor/{slug(goal)}-{short(base)}")

    return Snapshot(base=base, branch=branch)

# rollback is `git reset --hard {base}` — cheap, total, and available
# on the run where it is actually needed
```

07 Verify, roll back, and the CI/CD fit

8:27

> The agent writes and runs tests and rebuilds the project. If a test breaks it rolls back and goes round again — which is why the whole loop fits into a pipeline.

Once the patch is applied, the loop moves to verify. The agent writes and runs tests and rebuilds the project. Because something has now changed, it can show a diff against the starting snapshot, line by line. If a test breaks, it rolls back and the loop goes round again.

This loop is the answer to the question the whole thing opened with. The agentic probabilistic guesses do not go straight into the codebase. Think back to the obscure leap-year function: it probably would not have been removed, because either the human would not have approved that finding in the report, or a

failing test in the verify stage would have caught it. Two independent gates, and a guess has to pass both.

And because verification is just running tests and running builds, the whole loop slots into a CI/CD pipeline and can run alongside everyday development. The agentic AI is, in that framing, just another developer — one that opens a branch, makes changes, runs the suite, and puts up a pull request for review.

One caveat worth stating plainly, because it determines whether any of this works. The verify stage is only as strong as the test suite. On a module with thorough tests, this loop is genuinely safe. On a module with 20% coverage, verification passes and proves very little. The right sequencing on poorly tested code is therefore to have the agent add characterisation tests first, as a separate reviewed change, and only then refactor against them — otherwise you are asking the loop to verify a change using an instrument that cannot detect it.

And tests are not the only instrument available. Type checking catches signature drift. Linters catch obvious removals. A build catches broken imports. Contract tests catch external API changes. Each one is a cheap, independent gate, and stacking them raises the floor considerably on codebases where coverage is thin.

KEY POINTS

- → Verify runs the tests and rebuilds the project, then shows a diff against the snapshot.
- → A failing test triggers rollback and another pass through the loop.
- → Guesses face two independent gates: human approval and automated verification.
- → Because verification is tests and builds, the loop fits into CI/CD like another developer.
- → Verification is only as strong as the test suite — add characterisation tests first where coverage is thin.
- → Type checks, linters, builds and contract tests are additional cheap gates.

Verification as a stack of independent gates

Each gate catches a different class of mistake and each is cheap. On codebases where coverage is weak, the gates other than tests are what keep the floor off the ground.

```
def verify(snapshot):
    checks = [
        ("build",     lambda: run("make build")),        # imports, syntax
        ("types",     lambda: run("mypy .")),            # signature drift
        ("lint",      lambda: run("ruff check .")),      # obvious removals
        ("unit",      lambda: run("pytest tests/unit")), # behaviour
        ("contract",  lambda: run("pytest tests/contract")),  # external API
    ]

    for name, check in checks:
        if not check():
            git.reset_hard(snapshot.base)              # roll back
            return Failure(stage=name)                 # and loop again

    return Success(diff=git.diff(snapshot.base))
```

Tests first where coverage is thin

Two changes rather than one, and the ordering is not optional. Refactoring against a suite that cannot detect a behaviour change is refactoring without verification, whatever the pipeline reports.

```
coverage on billing/ : 21%

# wrong: refactor now. verify passes, and proves nothing.

# right, as two separately reviewed changes:
#   PR 1  agent writes characterisation tests against current behaviour.
#         a human reviews them for whether they capture what matters.
#         nothing is refactored.
#   PR 2  agent refactors. PR 1's tests are the instrument that verifies it.

# PR 1 is reviewable precisely because no behaviour changed in it.
```

The loop as a pipeline job

Once refactoring is expressed as branch, change, test, pull request, it is indistinguishable from any other contributor in your process — including being subject to the same review requirements.

```
on:
  schedule: [cron: "0 3 * * 1"]        # Monday, before anyone is online

jobs:
  refactor:
    steps:
      - run: agent refactor --goal "remove duplication in billing/" \
                            --max-findings 5 \
                            --require-tests-pass
      - run: gh pr create --title "Automated refactor: billing/" \
                          --body-file findings.md

# a branch, a diff, a description, and a human reviewer.
# the same process every other change goes through.
```

08 Deterministic patching with syntax trees

9:53

> The patch step can skip probabilistic generation entirely by parsing the code into a tree and operating on its structure.

There is a refinement to the patch step worth knowing about, because it removes the guessing from the part of the loop where guessing is most dangerous. Some tools parse the code into a structured tree — an abstract syntax tree, or a richer version that also tracks types and formatting — and make changes as operations on that tree.

Take a rename. On a tree, the operation is: find the symbol, update every reference to it. That is deterministic rather than probabilistic. The tool is not predicting what the renamed code should look like; it is walking a structure and applying a transformation that is correct by construction.

The difference from text manipulation is the important part. A find-and-replace on the string `state` will hit the variable you meant, an unrelated field on a different object, a substring inside `estate`, and a word in a comment. A tree operation distinguishes all of these because it knows what each node is, and a type-aware tree distinguishes two identically named fields on different classes, which plain syntax cannot.

The reason the richer, lossless variety matters: a plain AST discards formatting and comments, so regenerating source from it reformats the whole file and produces an enormous diff that buries the actual change. A lossless tree preserves the original text exactly, so the diff contains only what you changed — which is the difference between a reviewable pull request and one that gets approved unread.

The practical upshot is a division of labour. Let the model decide what to change, since that needs judgement about the codebase. Let the tree perform the change, since that needs exactness. Renames, signature changes, import rewrites, moving a function between modules, mechanical API migrations — all of these have deterministic implementations, and using them removes an entire class of failure.

KEY POINTS

- → Parsing to a syntax tree makes transformations structural rather than textual.
- → A rename becomes: find the symbol, update every reference — correct by construction.
- → Text replacement cannot distinguish variables, fields, substrings and comments; a tree can.
- → A type-aware tree distinguishes identically named symbols on different types.
- → Lossless trees preserve formatting and comments, keeping diffs small and reviewable.
- → Let the model decide what to change; let the tree perform the change.

Four things text replacement cannot tell apart

Every line contains the string. Only one should change. This is why mechanical refactorings performed by text substitution produce bugs that survive review — the wrong edits look exactly like the right ones.

```
customer_state = order.state          # the variable — rename this
shipping.state = "CA"                 # a different object's field
real_estate_value = 100               # a substring of another identifier
# the state machine handles this      # a comment
settings["state"] = active            # a string key

# sed 's/state/status/g' changes all five.
# a tree operation changes one.
```

A deterministic rename over a lossless tree

libcst preserves every byte of formatting and every comment, so the resulting diff contains the rename and nothing else. The visitor only fires on Name nodes, so comments, strings and substrings are untouched by construction.

```
import libcst as cst

class Rename(cst.CSTTransformer):
    def __init__(self, old, new):
        self.old, self.new = old, new

    def leave_Name(self, node, updated):
        # only identifier nodes reach here — never comments, strings
        # or substrings of other identifiers
        if updated.value == self.old:
            return updated.with_changes(value=self.new)
        return updated

source = Path("billing/charge.py").read_text()
tree = cst.parse_module(source)
out = tree.visit(Rename("temp2", "customer_state")).code

# formatting, comments and blank lines are byte-identical.
# the diff shows the rename and nothing else.
```

Which half does what

The division that makes the whole loop safer. The model contributes judgement, which is what it is good at; the tree contributes exactness, which the model cannot provide.

```
the model decides                the tree executes
─────────────────────────────────────────────────────────────
which variables are badly named  the rename, across every reference
what the new name should be      updating imports and call sites
which blocks are duplicated      extracting the shared function
whether a change is worthwhile   moving it between modules
how to explain the finding       preserving formatting and comments

# judgement is probabilistic. execution does not have to be.
```

09 Learning from accept, reject, pass and fail

10:35

> Every approval, rejection, passing test and failing test is a training signal, so the suggestions improve over time.

One more property of the loop. Every fix in the verify step that gets accepted or rejected is a training signal, and every test case that passes or fails is a training signal too. Models get trained to favour the changes that get accepted and pass, so the suggestions improve over time.

What makes this valuable is the quality of the signal. Most machine learning problems struggle for labels, and here the loop generates them as a by-product of ordinary work. A human approved this finding and rejected that one. This patch made the suite pass; that one broke three tests. Nobody was asked to label anything, and the labels are about exactly the thing you care about.

The test signal in particular is unusually clean. It is objective, automatic, and available at volume — closer to a programmatic grader than to a subjective preference judgement, which is what makes it usable for training rather than merely for dashboards.

Two cautions are worth carrying. The signal reflects what your reviewers approve, which includes their blind spots — if your team consistently waves through deletions, that is what gets reinforced. And it reflects your test suite's coverage: patches that break untested behaviour are labelled successes, so a weak suite teaches the model that changes it cannot detect are fine.

Which brings the lesson back to where it started. AI code refactoring can genuinely help pay down technical debt across a large codebase — but it has to be done in a way that verifies the output, and, at least for now, keeps humans in the loop. The verification and the humans are not friction around the capability. They are what makes the capability usable at all, and they are also what makes it get better.

KEY POINTS

- → Accept/reject decisions and test pass/fail results are both training signals.
- → The loop produces labels as a by-product of ordinary work, with no annotation effort.
- → Test outcomes are objective, automatic and high-volume — closer to a grader than a preference.
- → The signal inherits your reviewers' blind spots, including a habit of approving in bulk.
- → It also inherits your coverage: undetectable breakage is labelled as success.
- → Verification and human review are what make the capability usable, and what make it improve.

Labels generated by using the system

Four rows, four labelled examples, and nobody was asked to annotate anything. The last row is the most informative kind: a change that looked right, was approved by a human, and was caught by the suite.

```
finding                          approved?   patched?   tests
──────────────────────────────────────────────────────────────
bare except in charge.py         yes         yes        pass    +
duplicated day-count logic       yes         yes        pass    +
extract 180-line function        no          —          —       -
remove "unused" helper           yes         yes        FAIL    --

# the last row is worth the most: plausible, human-approved,
# and objectively wrong. that is a hard negative you cannot buy.
```

What the feedback loop inherits

Worth auditing occasionally, because both failure modes are self-reinforcing. A team that approves in bulk trains a model to propose more of what gets approved in bulk.

```
if reviewers approve everything at 5pm on Friday
  → the model learns that everything is acceptable

if coverage is 20%
  → patches that break untested behaviour are labelled successes
  → the model learns that undetectable breakage is fine

# the loop amplifies your review culture and your test suite,
# in whichever direction they already point.
```

Glossary

Refactoring

Changing a program's internal structure while leaving its external behaviour identical.

External behaviour

Everything observable from outside: return values, types, side effects, exceptions, emitted logs and timing. What a refactoring must preserve.

Technical debt

The compounding cost of deferred cleanup, paid as interest by every future change that has to route around the mess.

Characterisation test

A test asserting that behaviour has not changed rather than that it is correct. The right instrument before refactoring untested code.

Inline refactoring

Small, local suggestions made in the editor alongside the developer, reviewed continuously as they appear.

Agentic refactoring

An autonomous agent given a goal that plans and executes changes across the whole codebase, reviewed after the fact.

The refactoring loop

Plan, read, search, report, approve, snapshot, patch, verify — with rollback on failure and another pass round.

Snapshot

The recorded starting state, in practice a branch or commit, taken automatically so any change can be undone.

Abstract syntax tree (AST)

A structural representation of parsed code. Discards formatting and comments, so regenerating source reformats the file.

Lossless syntax tree

A richer tree that preserves formatting, comments and whitespace exactly, so a transformation produces a minimal diff.

Deterministic patching

Applying a change as a tree operation rather than generating text, making transformations like rename correct by construction.

Dead code

Code with no reachable callers. Hard to establish statically, because reflection, schedulers, flags and config can reach it invisibly.

Training signal

An outcome usable as a label. Here: whether a finding was approved, and whether the resulting patch passed the tests.

Check yourself

Answer first, then open each one.

Why does refactoring's definition — internal change, external behaviour preserved — make it a good first target for autonomous AI tools?

Because the success criterion is mechanically checkable. You are not asking whether the new code is good, only whether it behaves identically, and tests, types and builds can answer that automatically. A task with an automatic correctness check is one where a probabilistic system can be allowed to guess, because the guess gets verified before it counts.

The agent removed a function with no callers, no tests and no imports, and it was needed. Where exactly did the reasoning fail?

It did not fail. Every step was locally correct; the justification for keeping the function existed outside the repository, in a config-driven dispatcher or an incident three years ago. That is why the loop needs stages beyond reading code — searching dependencies and dynamic references, plus a human who might recognise the name.

Why does human approval sit between the report and the patch rather than after the patch?

Because it puts the review before the irreversible step, and it makes the review a ranked list with reasoning rather than a diff across hundreds of files. The expensive autonomous analysis happens unsupervised, which is fine because it changes nothing; the human's attention is spent only on the decision that has consequences.

Your module has 21% coverage. Why does a passing verify stage prove almost nothing, and what should happen first?

Verification detects behaviour changes only where tests observe behaviour, so most of the module's behaviour could change without any test noticing. The correct sequence is two separate reviewed changes: first the agent writes characterisation tests capturing current behaviour, which a human reviews; then it refactors against them. PR 1 is easy to review precisely because nothing behavioural changed in it.

Why is renaming a variable via a lossless syntax tree better than a careful find-and-replace, even when the replacement is scoped to one file?

Because text has no idea what any occurrence is. The same string appears as your variable, as a different object's field, inside a longer identifier, in a comment and as a dictionary key — and the wrong edits look exactly like the right ones in review. A tree operates on identifier nodes, so it changes only the symbol, and a lossless tree keeps the rest of the file byte-identical so the diff shows nothing else.

The loop generates training signals from approvals and test results. What does that signal inherit that you should audit?

Your review culture and your coverage. If reviewers approve in bulk, the model learns that most proposals are acceptable and proposes more of the same. If coverage is thin, patches that break untested behaviour are labelled as successes, teaching the model that undetectable breakage is fine. Both failure modes reinforce themselves.

Where to go next

- → Take a module you are afraid to change, have an agent write characterisation tests against its current behaviour, and read them — the tests alone usually reveal behaviour nobody knew was there.
- → Write a libcst transformer for a mechanical migration you have been putting off, and compare its diff against what a text-based script produces on the same repository.

- → Run an agentic refactoring with a deliberately vague goal on a scratch branch and read the report without approving anything; the findings tell you as much about the codebase as about the tool.
- → Audit your repository against the dead-code checklist and count how many entry points are invisible to static analysis — the number determines how much you can trust automated deletions.
- → Wire the loop into CI on a schedule with a hard cap on findings per run, and measure how many of its pull requests get merged unchanged over a month.
- → Compare the diff size of an AST-based transformation against a lossless-tree one on the same change, and notice how much reviewability depends on that difference.
- → Read Fowler's *Refactoring* for the catalogue of named transformations — agentic tools implement these, and knowing the names makes their reports far easier to evaluate.
